

A Computational Manual for the Global Projection Solution

Chebyshev–Smolyak projection methods, quadrature over a discrete default
fork,
and the solver architecture of `code/global/solver_recursive/`

Computational appendix

August 17, 2026

Abstract

This manual explains, from first principles and in full detail, how the two-country monetary-union model with sovereign risk and constrained financial intermediaries is solved *globally* by a Chebyshev–Smolyak projection method. It is written for economists who are comfortable with structural macroeconomics — Euler equations, Bellman equations, market clearing — but who have never implemented a non-linear global solution. Part 1 develops the mathematics of projection methods: what a projection method *is*, why Chebyshev polynomials and Chebyshev-extrema nodes rather than any other basis and any other points, and how Smolyak sparse grids defeat the curse of dimensionality in a seven-dimensional state space. Part 2 shows how the model’s theoretical equations become a concrete residual function $R(\mathbf{x}; \mathbf{S}, \Gamma) = 0$, and how expectations over a continuous risk factor *and* a discrete default fork are computed by Gauss–Hermite quadrature. Part 3 maps every mathematical object onto the exact file, function and variable that implements it, with code excerpts. Part 4 — the longest — documents the “under the hood” machinery: time iteration, damping, homotopy continuation, frozen continuations, log-space collocation, smoothed guards, failed-point masking, the Newton solver and its Jacobian, vectorisation, JIT compilation, and how solution accuracy is measured. Throughout, choices that were tried and *rejected* are documented alongside those that were kept, because in this class of problem the failures are as informative as the successes.

Companion documents. The file `model_equations.pdf` states the structural environment: preferences, technology, the intermediary problem, the government, and the equilibrium definition. The note `global_solver_methods.pdf` is a gentler and less complete introduction to the same numerical ideas; this manual is the technical reference and supersedes it where the two overlap.

Contents

Part 0. Orientation	4
1 What is being solved, and why globally	4
1.1 The object of interest	4
1.2 Why not linearise?	4
1.3 Road map	5
Part 1. Mathematical foundations	6
2 Projection methods from first principles	6
2.1 The general problem	6
2.2 What “choose a basis” means in practice	6
3 Chebyshev approximation in one dimension	7
3.1 Definition and the recurrence	7
3.2 Why Chebyshev: three properties that matter	7
3.3 Node placement and the Runge disaster	8
3.4 Fitting is a linear solve	9
3.5 Never extrapolate	9
4 Seven dimensions without the explosion	10
4.1 The curse of dimensionality	10
4.2 The Smolyak construction	10
4.3 What it buys	11
4.4 Anisotropic refinement	12
5 The collocation box	12
5.1 Mapping the economic domain to $[-1, 1]^d$	12
5.2 Why these bands, specifically	13
5.3 Invariance and the rotated box	13
5.4 The alternative: ergodic-set grids	14
Part 2. From theory to the residual function	15
6 Assembling the residual	15
6.1 What gets parameterised	15
6.2 The seven equilibrium conditions	15
6.3 Structure of one residual evaluation	16
6.4 The occasionally-binding constraint inside the residual	16
7 Computing expectations: quadrature over a continuum and a fork	17
7.1 Gauss–Hermite quadrature	17
7.2 The probabilists’ versus physicists’ trap	17
7.3 Mixing a continuous shock with a discrete fork	18
7.4 Evaluating the continuation	18
7.5 The honoured branch as a blend	19
7.6 Quadrature order	19
Part 3. Code architecture	21

8	Architecture	21
8.1	Module map	21
8.2	The RuleSet abstraction	21
8.3	Log-space collocation	22
9	Mathematics-to-code map	23
	Part 4. Computational tricks	25
10	The outer algorithm: time iteration	25
10.1	Why time iteration rather than one big Newton	25
10.2	The sweep	25
10.3	Damping and the convergence test	26
11	The inner solver	26
11.1	Powell’s hybrid method	26
11.2	Dual starts and graceful failure	26
12	Continuation and homotopy	27
12.1	Breaking the simultaneity knot	27
12.2	Warm-start chaining across regimes	28
12.3	Homotopy in the recovery rate	28
13	Robustness of the fit	29
13.1	Failed nodes must not enter the fit	29
14	Smoothing, guards and bounds	29
14.1	The smoothing primitives	29
14.2	Calibrating ϵ : the two-sided argument	30
14.3	Inventory of guards	30
15	Vectorisation and low-level performance	30
15.1	Where the time goes	30
15.2	Vectorisation	32
15.3	JIT compilation with an exact fallback	32
15.4	The spawn guard	32
16	Diagnosing the solution	33
16.1	Why residuals at the nodes are not enough	33
16.2	Euler errors on the ergodic set	33
16.3	Box escapes	33
16.4	Reading impulse responses off the rules	34
17	The known failure: why $\ell = 2$ does not converge	34
A	Reproducing the results	35
B	Glossary of numerical terms	35

Part 0. Orientation

1 What is being solved, and why globally

1.1 The object of interest

The model is a recursive competitive equilibrium in which every equilibrium quantity is a *function* of an aggregate state. Let

$$\mathbf{S}_t = (K_{D,t}, K_{F,t}, P_{D,t}, P_{F,t}, B_{D,t}, s_t, Z_{D,t}) \in \mathbb{R}^7, \quad d_t \in \{0, 1\}, \quad (1.1)$$

collect the two capital stocks, the two intermediaries' gross deposit obligations, the risky sovereign debt stock, the latent sovereign-risk factor and total factor productivity, together with a discrete default regime d . We must find functions

$$\mathbf{x}(d, \mathbf{S}) = (N_D, N_F, K'_D, K'_F, r_D, r_F, p)(d, \mathbf{S}) \quad (1.2)$$

— hours in each country, next period's capital in each country, the two deposit rates and the terms of trade — along with valuation functions $\varphi_X(d, \mathbf{S})$ (the intermediary's marginal value of net worth), $q^X(d, \mathbf{S})$ (sovereign bond prices), and household aggregates $C_X(d, \mathbf{S})$, $A_X(d, \mathbf{S})$, such that at *every* point of the state space the seven equilibrium conditions of Table 4 hold.

The unknowns are therefore not numbers but **functions on** $\mathbb{R}^7 \times \{0, 1\}$. Since a computer cannot store an arbitrary function, the first task of any global method is to replace each unknown function by a finite list of coefficients. That is what a projection method does.

1.2 Why not linearise?

The overwhelmingly standard alternative — perturbation, i.e. a Taylor expansion of the equilibrium conditions around the deterministic steady state — is *invalid* for this model, for three independent reasons. Each is worth stating carefully, because each is the reason a reader from a linear-DSGE background should expect the machinery in this manual to be necessary rather than ornamental.

(i) The incentive constraint is occasionally binding. The intermediary's leverage constraint $\varphi_{X,t} n_{X,t} \geq \lambda_X \mathcal{A}_{X,t}$ holds with complementary slackness,

$$\mu_{X,t} \geq 0, \quad \text{slack}_{X,t} \geq 0, \quad \mu_{X,t} \cdot \text{slack}_{X,t} = 0. \quad (1.3)$$

The equilibrium map is therefore only piecewise smooth: it has a *kink* on the manifold $\{\mu = 0\}$. A Taylor expansion around a point on one side of the kink extrapolates the wrong branch on the other side, and no order of expansion repairs this, because the map is not analytic there.

(ii) The mechanism of interest is a risk premium, which is identically zero at first order. Certainty equivalence holds in a first-order approximation: the linearised decision rules of a model with $\sigma_s = 0.63$ are numerically identical to those with $\sigma_s = 0$. But the entire object this model exists to measure — the pass-through of sovereign risk — *is* a risk premium. In the bond-pricing condition

$$q_t^D = \frac{\mathbb{E}_t[\Omega_{t,t+1} \Xi_{t+1}^D]}{\mathbb{E}_t[\Omega_{t,t+1}] R_{D,t} + \lambda_D \mu_{D,t}} = \frac{\mathbb{E}_t[\Xi_{t+1}^D]}{R_{D,t}} + \frac{\text{Cov}_t(\Omega_{t,t+1}, \Xi_{t+1}^D)}{\mathbb{E}_t[\Omega_{t,t+1}] R_{D,t}} - (\text{constraint discount}), \quad (1.4)$$

the covariance term is second order in the shock and vanishes under certainty equivalence. A first-order solution would price the risky sovereign bond at its actuarially fair value and report a zero risk premium — i.e. it would return exactly nothing.

(iii) **The relevant experiment is a large, rare, discrete event.** The default fork is a jump to a state in which the sovereign claim is written down to 45% of face value. The economy in that branch is far from the steady state: measured on the solved rules, the intermediary's marginal value of net worth φ reaches more than thirteen times its steady-state level in the deep default corners. A local approximation is not merely inaccurate there; it is qualitatively wrong, and — as §17 documents — getting the sign of the output response right depends on representing that region.

► *These three reasons compound. A first-order solution gives no premium; a second- or third-order perturbation gives a premium but still smooths the occasionally-binding kink and still expands around a point far from the default branch. A global method is the only approach that represents both the kink and the distant branch — which is why the entire perfect-foresight and representative-branch machinery this project began with was deleted in favour of the projection solver.*

1.3 Road map

- **Part 1** (§2–§5) — projection methods, Chebyshev bases, Smolyak grids, and the collocation box.
- **Part 2** (§6–§7) — the residual function and numerical integration over the risk factor and the default fork.
- **Part 3** (§8–§9) — code architecture and the mathematics-to-code map.
- **Part 4** (§10–§16) — the solver algorithm and every computational trick it relies on.

Part 1. Mathematical foundations of the solution method

2 Projection methods from first principles

2.1 The general problem

Every global solution method starts from the same abstract statement. We seek an unknown function $g : \mathcal{D} \rightarrow \mathbb{R}^n$, defined on a domain $\mathcal{D} \subseteq \mathbb{R}^d$, that satisfies a functional equation

$$\mathcal{H}[g](\mathbf{S}) = 0 \quad \text{for all } \mathbf{S} \in \mathcal{D}, \quad (2.1)$$

where $\mathcal{H}$ is an operator that may involve g evaluated at other points (through the transition law) and expectations over future shocks. In our case $\mathcal{H}$ is the stack of seven equilibrium conditions and g is the collection of decision rules (1.2).

Equation (2.1) is an infinite system: one vector equation for each of the uncountably many points $\mathbf{S} \in \mathcal{D}$. A **projection method** makes it finite in two steps.

Step 1: parameterise. Choose a family of basis functions $\{\Psi_1, \dots, \Psi_n\}$ and approximate

$$g(\mathbf{S}) \approx \hat{g}(\mathbf{S}; c) = \sum_{b=1}^n c_b \Psi_b(\mathbf{S}), \quad (2.2)$$

so the unknown function is replaced by the unknown coefficient vector $c \in \mathbb{R}^n$.

Step 2: project. Substituting (2.2) into (2.1) defines the *residual function*

$$R(\mathbf{S}; c) \equiv \mathcal{H}[\hat{g}(\cdot; c)](\mathbf{S}), \quad (2.3)$$

which will not be identically zero for any finite n . We instead require it to be zero in a weighted-average sense: choose n weighting functions $w_1, \dots, w_n$ and impose

$$\int_{\mathcal{D}} w_b(\mathbf{S}) R(\mathbf{S}; c) d\mathbf{S} = 0, \quad b = 1, \dots, n. \quad (2.4)$$

This is n equations in n unknowns.

Different choices of w_b give the classical variants: Galerkin ($w_b = \Psi_b$), least squares ($w_b = \partial R / \partial c_b$), and — the one used here — **collocation**, where $w_b(\mathbf{S}) = \delta(\mathbf{S} - \mathbf{S}_b)$ is a Dirac mass at a selected node $\mathbf{S}_b$. Collocation reduces (2.4) to the simple and intuitive requirement

$$\boxed{R(\mathbf{S}_b; c) = 0, \quad b = 1, \dots, n} \quad (2.5)$$

the residual is exactly zero at n chosen points of the state space. Between those points the residual is not zero; how large it is there is precisely the accuracy question of §16.

Remark 2.1 (Why collocation). Galerkin projection requires integrating the residual against each basis function over a seven-dimensional domain, and each residual evaluation here costs a seven-equation non-linear solve with a multi-branch quadrature inside it. Collocation requires the residual only at n points. With $n = 259$ nodes and two regimes, the cost difference is the difference between a solve that runs in ninety minutes and one that does not run at all.

2.2 What “choose a basis” means in practice

Two decisions remain: *which* basis functions Ψ_b , and *which* collocation points $\mathbf{S}_b$. These are not independent — a bad pairing produces a near-singular collocation matrix or a wildly oscillating interpolant — and the next two sections address them together. The answer used here is: tensor products of Chebyshev polynomials, evaluated at Chebyshev-extrema nodes, both restricted to a Smolyak sparse index set.

3 Chebyshev approximation in one dimension

3.1 Definition and the recurrence

The Chebyshev polynomials of the first kind are defined on $[-1, 1]$ by

$$T_k(x) = \cos(k \arccos x), \quad k = 0, 1, 2, \dots \quad (3.1)$$

The trigonometric definition makes several properties immediate: $|T_k(x)| \leq 1$ on $[-1, 1]$; T_k has exactly k roots and $k + 1$ extrema in $[-1, 1]$; and T_k oscillates between $+1$ and -1 with equal amplitude — the equioscillation property that underlies its optimality. Despite the cosine, T_k is a polynomial of degree k , as the three-term recurrence shows:

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{k+1}(x) = 2x T_k(x) - T_{k-1}(x). \quad (3.2)$$

Equation (3.2) — not (3.1) — is what the code evaluates, because it is $O(k)$ per point, numerically stable, and free of transcendental function calls. It is implemented verbatim:

```

1 def chebyshev_basis_1d(x, max_deg):
2     # T_0..T_max_deg AT POINTS x VIA THE RECURRENCE (SHAPE (len(x), max_deg+1)).
3     x = np.asarray(x, dtype=float)
4     T = np.empty((x.size, max_deg + 1))
5     T[:, 0] = 1.0
6     if max_deg >= 1:
7         T[:, 1] = x
8     for k in range(2, max_deg + 1):
9         T[:, k] = 2.0 * x * T[:, k - 1] - T[:, k - 2]
10    return T

```

Listing 1: The Chebyshev recurrence. `state.grid.py`, `chebyshev.basis_1d`.

The polynomials are orthogonal on $[-1, 1]$ with respect to the weight $(1 - x^2)^{-1/2}$:

$$\int_{-1}^1 \frac{T_j(x) T_k(x)}{\sqrt{1 - x^2}} dx = \begin{cases} 0, & j \neq k, \\ \pi, & j = k = 0, \\ \pi/2, & j = k \geq 1. \end{cases} \quad (3.3)$$

Orthogonality is what makes the coefficients of a Chebyshev expansion decay rapidly and independently, so that truncating the series at degree n discards a well-controlled remainder rather than redistributing error across the retained terms.

3.2 Why Chebyshev: three properties that matter

(a) Near-minimax accuracy. Let f be continuous on $[-1, 1]$ and let p_n^* be its best degree- n polynomial approximation in the sup norm. The degree- n Chebyshev interpolant p_n satisfies

$$\|f - p_n\|_\infty \leq (1 + \Lambda_n) \|f - p_n^*\|_\infty, \quad \Lambda_n \sim \frac{2}{\pi} \log n, \quad (3.4)$$

where Λ_n is the Lebesgue constant of the node set. The logarithmic growth is the crucial fact: at $n = 64$ the Chebyshev interpolant is within a factor of about 3.6 of the unimprovable approximation. For *equispaced* nodes the Lebesgue constant instead grows like $2^n / (e n \log n)$ — exponentially — which is the Runge phenomenon of §3.3.

(b) Spectral (geometric) convergence for smooth functions. If f is analytic in the Bernstein ellipse E_ρ (the ellipse in the complex plane with foci ± 1 and semi-axis sum $\rho > 1$), then

$$\|f - p_n\|_\infty \leq \frac{2M\rho^{-n}}{\rho - 1}, \quad M = \max_{z \in E_\rho} |f(z)|, \quad (3.5)$$

i.e. the error falls *geometrically* in the degree. Adding one degree multiplies the accuracy by a constant factor, rather than adding a fixed number of digits as in a finite-difference scheme. If instead f has only k continuous derivatives, convergence degrades to the algebraic rate $O(n^{-k})$.

★ *Property (b) is exactly what the occasionally-binding constraint threatens. The equilibrium map is smooth on each side of $\{\mu = 0\}$ but only C^0 across it. Where the kink lies inside the collocation box, the geometric rate (3.5) is lost and the interpolant develops Gibbs-type oscillations. Section 17 shows this is not hypothetical: it is the reason the solve converges at approximation level $\ell = 1$ and fails at $\ell = 2$.*

(c) **A well-conditioned collocation matrix.** Because the basis is orthogonal and the nodes are the natural ones for that basis, the matrix Φ of §3.4 is far better conditioned than the Vandermonde matrix of a monomial basis $\{1, x, x^2, \dots\}$, whose condition number grows exponentially in degree. In double precision a monomial basis becomes unusable above degree ten or so; the Chebyshev basis remains usable into the hundreds.

3.3 Node placement and the Runge disaster

Interpolation error at any x factors as

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^n (x - x_j), \quad (3.6)$$

so the node placement enters only through the *node polynomial* $\prod_j (x - x_j)$. Minimising its sup norm over node choices is a classical problem whose solution clusters points towards the endpoints with the arcsine density $1/(\pi\sqrt{1-x^2})$. Equispaced nodes leave the node polynomial exponentially large near ± 1 , and interpolants of even innocuous functions — Runge’s $f(x) = 1/(1+25x^2)$ is the textbook case — *diverge* as n grows.

This code uses the **Chebyshev extrema** (Gauss–Lobatto) points,

$$x_j = -\cos\left(\frac{\pi j}{m-1}\right), \quad j = 0, 1, \dots, m-1, \quad (3.7)$$

which include the endpoints ± 1 and satisfy the arcsine clustering. The choice of extrema rather than *roots* (Gauss points) is deliberate and is what makes the Smolyak construction of §4 possible: extrema sets are **nested**, so that the level- i point set is a subset of the level- $(i+1)$ set. Root sets are not.

```

1 def _level_points(i):
2     # FULL 1D CHEBYSHEV-EXTREMA SET OF LEVEL i (m = 1, 3, 5, 9, ... POINTS).
3     if i == 1:
4         return np.array([0.0])
5     m = 2 ** (i - 1) + 1
6     return -np.cos(np.pi * np.arange(m) / (m - 1))
7
8 def _new_points(i):
9     # POINTS INTRODUCED AT LEVEL i (DISJOINT ACROSS LEVELS BY NESTEDNESS).
10    if i == 1:
11        return np.array([0.0])
12    if i == 2:
13        return np.array([-1.0, 1.0])
14    return _level_points(i)[1::2] # odd positions are absent from level i-1

```

Listing 2: Nested Chebyshev-extrema levels. `state_grid.py`.

The level sequence is $m_i = 1, 3, 5, 9, 17, \dots$ (i.e. $m_1 = 1$ and $m_i = 2^{i-1} + 1$ for $i \geq 2$), and the “new points” at level i are the odd-indexed members of level i , which are exactly those absent from level $i - 1$. Table 1 lists the first few levels.

Table 1: Nested one-dimensional levels: points and the degrees they introduce.

Level i	m_i (cumulative points)	New points	New degrees
1	1	$\{0\}$	$\{0\}$
2	3	$\{-1, +1\}$	$\{1, 2\}$
3	5	$\{-\frac{\sqrt{2}}{2}, +\frac{\sqrt{2}}{2}\}$	$\{3, 4\}$
4	9	4 points	$\{5, 6, 7, 8\}$
5	17	8 points	$\{9, \dots, 16\}$

The pairing of “new points” with “new degrees” in the last column is what keeps the collocation system square, and is implemented as:

```

1 def _new_degrees(i):
2     # CHEBYSHEV DEGREES INTRODUCED AT LEVEL i (|new degrees| = |new points|).
3     if i == 1:
4         return np.array([0])
5     if i == 2:
6         return np.array([1, 2])
7     m_prev = 2 ** (i - 2) + 1
8     return np.arange(m_prev, 2 ** (i - 1) + 1)

```

Listing 3: Degrees introduced at each level. `state_grid.py`, `_new_degrees`.

3.4 Fitting is a linear solve

Given nodes $\{x_p\}_{p=1}^n$ and degrees $\{k_b\}_{b=1}^n$, build the collocation matrix

$$\Phi_{pb} = T_{k_b}(x_p), \quad \Phi \in \mathbb{R}^{n \times n}, \quad (3.8)$$

and let $y_p = f(x_p)$ be the function values. The interpolation conditions $\hat{f}(x_p) = y_p$ are the linear system

$$\Phi c = y \quad \implies \quad c = \Phi^{-1} y. \quad (3.9)$$

Two implementation points matter. First, Φ depends only on the *grid*, not on the function being fitted, so it is built once and LU-factorised once; every subsequent fit is a triangular back-substitution costing $O(n^2)$ instead of $O(n^3)$. Second, this is what makes it cheap to fit *seventeen* rules in two regimes on every sweep: they all share one factorisation.

```

1 self._Phi = self._basis_unit(self.points_unit)      # (n, n) collocation basis
2 self._lu = lu_factor(self._Phi)
3 ...
4 def fit(self, values):
5     # COLLOCATION COEFFICIENTS FROM VALUES AT self.points ((n,) OR (n, k)).
6     return lu_solve(self._lu, np.asarray(values, dtype=float))

```

Listing 4: One factorisation, many fits. `state_grid.py`, `SmolyakGrid`.

3.5 Never extrapolate

Equation (3.5) holds *inside* the interpolation interval. Outside it, the Chebyshev interpolant behaves like the leading monomial $x^{k_{\max}}$ and diverges rapidly: at $x = 1.3$ with $k_{\max} = 8$, T_8 is already of order 10^3 . Every evaluation of the decision rules must therefore be inside the box. The code enforces this explicitly at the one place where an off-box state can arise — the continuation point in the expectation — by clipping:

$$\mathbf{S}'_{\text{eval}} = \Pi_{[\text{lo}, \text{hi}]}(\mathbf{S}'), \quad \text{code: } \text{cont.eval_all}(\text{d.next}, \text{cont.grid.clip}(\text{Sn})). \quad (3.10)$$

Clipping is not free — §5 shows that an over-tight box makes the clip distort the risk process itself — so the box must be wide enough that clipping is rare, and how often it binds is reported by the accuracy diagnostic (§16).

4 Seven dimensions without the explosion

4.1 The curse of dimensionality

The natural multi-dimensional extension of §3 is the tensor product: take m nodes per dimension and use all m^d combinations, with the corresponding product basis $\prod_j T_{k_j}(x_j)$. In $d = 7$ with the modest choice $m = 3$ this is $3^7 = 2,187$ points; with $m = 5$ it is 78,125; with $m = 9$ it is 4,782,969. Each point requires a seven-equation non-linear solve with a $3M$ -node quadrature inside it, in each of two regimes. At roughly a second per point, $m = 5$ alone is a day of computation per sweep, and dozens of sweeps are needed. Tensor products are unusable here.

4.2 The Smolyak construction

Smolyak’s (1963) insight, in the nested Chebyshev form of Krueger and Kubler (2004), is that most of the tensor product is wasted: it spends points on *high-order interactions* — terms like $x_1^8 x_2^8 x_3^8$ — which contribute almost nothing to the accuracy of a smooth function. A sparse grid keeps the cheap combinations and discards the expensive ones, according to a budget on the total level.

Formally, index each dimension by a level $i_j \geq 1$, and define the admissible multi-index set

$$\mathcal{I}(\ell) = \left\{ i = (i_1, \dots, i_d) \in \mathbb{N}^d : \sum_{j=1}^d (i_j - 1) \leq \ell, \quad i_j - 1 \leq \ell_j \quad \forall j \right\}, \quad (4.1)$$

where ℓ is the **approximation level** (the code calls it `mu`; we write ℓ throughout to avoid collision with the incentive-constraint multiplier μ) and the optional per-dimension caps ℓ_j produce the anisotropic grids of §4.4. The grid and the basis are then unions of small tensor products over that index set, using the *new* point and degree sets of Table 1:

$$\mathcal{H}(\ell) = \bigcup_{i \in \mathcal{I}(\ell)} \Delta_{i_1} \times \Delta_{i_2} \times \dots \times \Delta_{i_d}, \quad \Delta_i = \text{new points at level } i, \quad (4.2)$$

$$\mathcal{B}(\ell) = \bigcup_{i \in \mathcal{I}(\ell)} \mathcal{K}_{i_1} \times \mathcal{K}_{i_2} \times \dots \times \mathcal{K}_{i_d}, \quad \mathcal{K}_i = \text{new degrees at level } i. \quad (4.3)$$

Because $|\Delta_i| = |\mathcal{K}_i|$ for every i and the unions are over the same index set, $|\mathcal{H}(\ell)| = |\mathcal{B}(\ell)| = n$: the collocation matrix is **square by construction**, and (3.9) carries over unchanged. This is the structural reason the “new points / new degrees” pairing of Table 1 is set up as it is.

The multi-index enumeration is a depth-first recursion over the remaining level budget:

```

1 def _multi_indices(d, mu, mu_vec):
2     # ALL LEVEL MULTI-INDICES i (EACH >= 1) WITH sum(i-1) <= mu AND i-1 <= mu_vec.
3     out = []
4     def rec(prefix, budget):
5         # DEPTH-FIRST ENUMERATION UNDER THE REMAINING LEVEL BUDGET.
6         j = len(prefix)
7         if j == d:
8             out.append(tuple(prefix))
9             return
10        for lev in range(1, min(budget, mu_vec[j]) + 2):
11            rec(prefix + [lev], budget - (lev - 1))
12    rec([], mu)
13    return out

```

Listing 5: Enumerating $\mathcal{I}(\ell)$. `state_grid.py`, `_multi_indices`.

and the grid assembly is a loop of small meshgrids:

```

1 pts, degs = [], []
2 for i_vec in _multi_indices(self.d, self.mu, self.mu_vec):
3     axes_p = [_new_points(i) for i in i_vec]
4     axes_d = [_new_degrees(i) for i in i_vec]
5     mesh_p = np.meshgrid(*axes_p, indexing="ij")
6     mesh_d = np.meshgrid(*axes_d, indexing="ij")
7     pts.append(np.column_stack([m.ravel() for m in mesh_p]))
8     degs.append(np.column_stack([m.ravel() for m in mesh_d]))
9 self.points_unit = np.vstack(pts)
10 self.degrees      = np.vstack(degs).astype(int)

```

Listing 6: Building the sparse grid and its basis degrees. `state_grid.py`, `SmolyakGrid.__init__`.

The multi-dimensional basis matrix is then the elementwise product of one-dimensional Chebyshev matrices, one per dimension — a fully vectorised operation over all points and all basis functions at once:

```

1 def _basis_unit(self, u):
2     # BASIS MATRIX AT UNIT-BOX POINTS: PRODUCTS OF PER-DIMENSION CHEBYSHEVS.
3     u = np.atleast_2d(u)
4     B = np.ones((u.shape[0], self.n))
5     for j in range(self.d):
6         Tj = chebyshev_basis_1d(u[:, j], self.max_deg)
7         B *= Tj[:, self.degrees[:, j]]
8     return B

```

Listing 7: The tensor basis, vectorised. `state_grid.py`, `_basis_unit`.

4.3 What it buys

Table 2 gives the actual point counts for $d = 7$, computed from the implementation. The savings are dramatic and grow with dimension.

Table 2: Smolyak versus tensor-product point counts in $d = 7$ (computed from `SmolyakGrid`).

Level ℓ	Max degree	Smolyak points	Tensor points	Ratio
1	2	15	2,187	1 : 146
2	4	113	78,125	1 : 691
3	8	589	4,782,969	1 : 8,120
<i>Grids actually used:</i>				
[1, 1, 1, 1, 1, 3, 0] (production, risk)	8	259	—	anisotropic
[0, 0, 1, 1, 0, 3, 0] (frozen capital)	8	41	—	anisotropic
$\ell = 1$ isotropic (TFP experiment)	2	15	2,187	1 : 146

The approximation-theoretic guarantee is that the sparse grid loses surprisingly little. Level ℓ reproduces *exactly* every polynomial of total degree $\leq \ell$ in d variables, including all cross terms. At $\ell = 2$ this means every complete quadratic — all d linear terms, all d squares and all $d(d-1)/2$ interactions — which is already the entire information content of a second-order perturbation, but represented globally rather than locally. This property is regression-tested directly:

```

1 def test_quadratic_exactness():
2     # LEVEL 2 REPRODUCES COMPLETE QUADRATICS INCL. CROSS TERMS (KK04 PROPERTY).
3     rng = np.random.default_rng(1); d = 8
4     g = SmolyakGrid(-np.ones(d), np.ones(d), mu=2)
5     A = rng.standard_normal((d, d)); A = 0.5 * (A + A.T)
6     b = rng.standard_normal(d); c0 = rng.standard_normal()
7     def f(x):
8         return np.einsum("ij,jk,ik->i", x, A, x) + x @ b + c0

```

```

9   coef = g.fit(f(g.points))
10  x = rng.uniform(-1, 1, size=(800, d))
11  assert np.max(np.abs(g.eval(coef, x) - f(x))) < 1e-9

```

Listing 8: The exactness property, tested. `tests/test_state_grid.py`.

The general error bound (Barthelmann, Novak and Ritter, 2000) for f with bounded mixed derivatives of order k is

$$\|f - \hat{f}_\ell\|_\infty = O\left(n^{-k}(\log n)^{(d-1)(k+1)}\right), \quad (4.4)$$

against the tensor-product rate $O(n^{-k/d})$ — the curse of dimensionality is reduced from an exponent to a logarithmic factor.

4.4 Anisotropic refinement

The isotropic budget in (4.1) spends resolution equally on all seven states. That is wasteful here, because the states are economically very unequal: the sovereign-risk factor s ranges over ± 4.35 in logit units and is where every non-linearity lives, whereas the two capital stocks move by a few percent over the ergodic set. Setting per-dimension caps ℓ_j — the argument `mu_vec` — concentrates points where the curvature is:

$$\ell_{\text{vec}} = \left[\underbrace{1, 1}_{K_D, K_F}, \underbrace{1, 1}_{P_D, P_F}, \underbrace{1}_{B_D}, \underbrace{3}_s, \underbrace{0}_{Z_D} \right] \implies n = 259, \quad k_{\max} = 8. \quad (4.5)$$

The risk factor gets degree-8 resolution; TFP, which is never shocked in the risk experiment, gets a single node (a rule flat in Z); the remaining five states get the minimum non-trivial refinement.

► *The anisotropic grid is not a cost-saving convenience — it changes the answer. On the isotropic $\ell = 1$ grid the long bond reprices by only -1.3% under the headline shock, because degree-2 resolution in s cannot represent the convexity of the price in the risk factor. With $\ell_s = 3$ it reprices by -4.8% , the bond loss crashes intermediary net worth, and the multiplier μ spikes endogenously. The mechanism the model exists to measure is invisible at uniform resolution.*

★ *The opposite error is equally consequential. The variant $[0, 0, 1, 1, 0, 3, 0]$ (41 points) freezes K_D , K_F and B_D at a single node, making the rules flat in capital and debt. This runs in ten minutes instead of ninety, but it switches off the capital-accumulation channel entirely: measured, the impact credit-spread response is $+143$ bp with capital live and -27 bp with it frozen. A grid choice silently deleted the mechanism and flipped a sign.*

5 The collocation box

5.1 Mapping the economic domain to $[-1, 1]^d$

Chebyshev theory lives on $[-1, 1]^d$; the model lives on an economic box $[\text{lo}, \text{hi}] \subset \mathbb{R}^7$. The affine map and its inverse are

$$u_j(\mathbf{S}) = 2 \frac{S_j - \text{lo}_j}{\text{hi}_j - \text{lo}_j} - 1, \quad S_j = \text{lo}_j + \frac{1}{2}(u_j + 1)(\text{hi}_j - \text{lo}_j), \quad (5.1)$$

implemented as `to_unit` and `from_unit`. The box is centred on the steady state with per-state bands:

$$\begin{array}{lll} K_X : \pm 2\%, & P_D : \pm 12\%, & P_F : \pm 20\%, \\ B_D : \pm 8\%, & s : \bar{s} \pm 4.35, & Z_D : \pm 3\%. \end{array} \quad (5.2)$$

5.2 Why these bands, specifically

Band selection is one of the highest-leverage choices in the whole solve, and the values in (5.2) are measured rather than guessed.

The s band. The half-width 4.35 is Bocola’s own coverage, equal to ± 2.16 unconditional standard deviations of the s process ($\sigma_s/\sqrt{1-\rho_s^2} = 2.02$). This must be checked against the *conditional* innovation, not just the unconditional spread, because the continuation states in the quadrature are clipped to the box: if the box is narrow in conditional standard deviations, the clip mean-reverts s harder than ρ_s does. An earlier calibration with $\sigma_s = 1.5075$ made the box only 1.4–2.3 conditional standard deviations wide, which cut the *effective* persistence of s from 0.95 to 0.80 and removed roughly 62% of the long bond’s repricing — silently. There is now a regression test for exactly this:

```

1      # and the clip must not distort the process in the ergodic core
2      x, w = np.polynomial.hermite_e.hermegauss(5)
3      w = w / w.sum()
4      grid_s = np.linspace(sp["s_star"] - unc, sp["s_star"] + unc, 21)
5      Ec = np.array([w @ np.clip((1 - sp["rho_s"]) * sp["s_star"] + sp["rho_s"] * s
6                        + sp["sigma_s"] * x, lo, hi) for s in grid_s])
7      rho_eff = np.polyfit(grid_s, Ec, 1)[0]
8      assert rho_eff > 0.93, f"box clip cuts effective rho_s to {rho_eff:.3f}"

```

Listing 9: Guarding against a box that distorts the shock process. `tests/test_state_grid.py`.

The P_F band. At the original $\pm 8\%$ the sweep degraded from about the tenth iteration and 22 of 41 points stopped clearing permanently, leaving the fit anchored on frozen cold-start values with a worst residual of 9.6×10^{-3} . At $\pm 20\%$ the entire grid clears to 1.1×10^{-14} with zero failures, and $4.5\times$ faster. The reason is economic: foreign-side wealth is a documented quasi-unit-root state whose one-step image ran to $+20.6\%$ against a $\pm 8\%$ box. **A box must be able to contain the image of its own points.**

The P_D band. Symmetric at $\pm 12\%$, because the period map has no solution above roughly $P_D + 15\%$. An asymmetric (0.20, 0.12) pair was tried and is *worse* (six failures): it shifts the box centre off the steady state, so the steady state stops being a collocation node — and a solution that is not exact at its own rest point is a poor starting point for everything else.

5.3 Invariance and the rotated box

Ideally the box would be *one-step invariant*: the image of every point under the transition law would lie inside it. Measured at the steady state, the Jacobian of the deposit-obligation block is

$$J = \frac{\partial(P'_D, P'_F)}{\partial(P_D, P_F)} = \begin{pmatrix} 1.076 & -0.310 \\ 2.016 & -1.252 \end{pmatrix}, \quad \text{eig}(J) = \{0.766, -0.942\}. \quad (5.3)$$

The dynamics are stable — both eigenvalues are inside the unit circle — but J is strongly non-normal (singular values 2.61 and 0.28). Invariance of an axis-aligned box requires $|J|b \leq b$ componentwise for some positive half-width vector b , where $|J|$ is the entrywise absolute value. Here $\rho(|J|) = 1.96 > 1$, so by Perron–Frobenius **no axis-aligned box centred on the steady state is one-step invariant, at any bandwidth**. On the eigenbasis, by contrast, the map is diagonal with $|\lambda| < 1$ and every box is invariant.

The code therefore supports collocating on rotated coordinates $z = R(\mathbf{S} - \bar{\mathbf{S}})$ (Bocola’s V -transform), with the box drawn on z while every public method still speaks natural coordinates:

```

1 def _fwd(self, x):
2     # NATURAL -> ROTATED BOX COORDINATES (identity when rot is None).
3     x = np.atleast_2d(x)
4     return x if self.rot is None else (x - self.centre) @ self.rot.T
5
6 def clip(self, x):
7     # PROJECT POINTS INTO THE BOX (SIMULATION USE; EXPLICIT, NOT SILENT).
8     # Clipping happens in the ROTATED coordinates the box is drawn on, so a
9     # rotated box projects onto its own faces, not onto an axis-aligned hull.
10    return self._bwd(np.clip(self._fwd(x), self.lo, self.hi))

```

Listing 10: Rotated box coordinates. `state_grid.py`, `SmolyakGrid`.

★ *The rotation is correct in theory and measures worse: 11 of 41 points fail and the worst residual is 1.2×10^{-2} , against zero failures and 1.1×10^{-14} for the axis-aligned box. The reason is instructive. A rotated box is a parallelogram in natural coordinates, and its corners reach P_F states at which the period map has no equilibrium at all. Invariance was simply not the binding constraint; feasibility of the collocation points was. The facility is kept, tested, and off by default (`ROTATE_P = False` in `main.py`).*

5.4 The alternative: ergodic-set grids

When the box approach fails outright — as it does at $\ell = 2$, where 40–50% of the collocated corners are infeasible — the structural fix is to stop collocating on a box at all. The `eds.py` module implements the Maliar–Maliar ε -distinguishable-set method: simulate the model to obtain the ergodic cloud, keep a well-spaced subset by farthest-point sampling, and fit by complete-polynomial least squares on that cloud. The solver then only ever visits states the model actually reaches. `EDSGrid` is a drop-in replacement exposing the same `points / n / d / fit / eval / basis / clip` interface, so `RuleSet`, `point_map` and the time-iteration driver are reused unchanged — an illustration of why the grid abstraction is worth the indirection.

Part 2. From theory to the residual function

6 Assembling the residual

6.1 What gets parameterised

Seventeen functions are approximated on the grid, in each of two default regimes — 34 coefficient vectors in total. They fall into three groups:

Table 3: The approximated rules. Source: `decision_rules.py`.

Group	Members	Role
SOLVE7	N_D, N_F, Kp_D, Kp_F, rdep_D, rdep_F, p	The pointwise Newton unknowns $\mathbf{x}$: solved at every node from the seven market-clearing residuals.
DERIVED (valuations)	alpha_D, alpha_F, Q_bD, Q_bF, r_wc_D, r_wc_F	Banker valuations φ_X , bond prices q^X and the lending rate: <i>read off</i> the Euler recursions given the frozen continuation, not solved for.
DERIVED (household)	C_D, C_F, A_D, A_F	Aggregate consumption and deposits, from the budget identity and quantity clearing.

The split between “solved” and “read off” is a design choice with real consequences, discussed in §12.1: solving for all seventeen jointly at each node would be a 17×17 non-linear system per point instead of 7×7 , and — more importantly — would reintroduce a simultaneity between the bond price and next period’s state that the frozen-continuation device removes.

6.2 The seven equilibrium conditions

At a node $(d, \mathbf{S})$, given a frozen continuation rule set Γ , the residual function is

$$R : \mathbb{R}^7 \rightarrow \mathbb{R}^7, \quad R(\mathbf{x}; d, \mathbf{S}, \Gamma) = 0, \quad (6.1)$$

with components as in Table 4. Each row states the economics, the residual as coded, and — crucially — the *normalisation*.

Table 4: The residual vector. Source: `point_map.py`, `point_residuals`.

#	Condition	Residual as coded	Pins
1	Capital Euler, D	$(\mathbb{E}[\Omega_D(r_D^{k'} - r_D)] - \lambda_K \mu_D) / \mathbb{E}[\Omega_D]$	K'_D
2	Capital Euler, F	$(\mathbb{E}[\Omega_F(r_F^{k'} - r_F)] - \lambda_K \mu_F) / \mathbb{E}[\Omega_F]$	K'_F
3	Labour market, D	$(\chi_D N_D^{1/\nu} - w_D / P_D^c) / (w_D / P_D^c)$	N_D
4	Labour market, F	$(\chi_F N_F^{1/\nu} - w_F / P_F^c) / (w_F / P_F^c)$	N_F
5	Deposit Euler, D	$x_D^{-\sigma} / (\tilde{\beta}_D \mathbb{E}[(1 + r_D^c) x_D'^{-\sigma}]) - 1$	r_D
6	Deposit-UIP	$(1 + r_D) - (1 + r_F) \mathbb{E}[p'] / p + \kappa_{\text{nfa}}(P_D - P_F) / \bar{Y}_D$	r_F
7	Goods market, D	$(Y_D - P_D^c C_D - I_D - N X_D - G_D) / \bar{Y}_D$	p

► **Normalisation is not cosmetic.** A Newton solver measures convergence by $\|R\|_\infty$, so residuals in wildly different units make the tolerance meaningless and the step direction dominated by whichever equation happens to be measured in the largest units. Every residual above is therefore rendered dimensionless: Euler conditions are divided by the discount kernel $\mathbb{E}[\Omega]$ so they read in return units, the labour conditions by the real wage, and the goods market by steady-state output. Note the choice not made: dividing the capital Euler by $\lambda_K \bar{\mu}$ instead of $\mathbb{E}[\Omega]$ would amplify it roughly $130\times$ relative to the others and wrecks the solver step.

6.3 Structure of one residual evaluation

A single call to `point_residuals` performs the following, in order. This is the innermost loop of the entire solve, executed hundreds of times per node per sweep, so its structure is worth reading carefully.

1. **Unpack** the trial vector $\mathbf{x}$ and the state $\mathbf{S}$.
2. **Firms and capital, current period.** Evaluate the production and capital blocks: output, the frictionless wage, mpk , Tobin's q , investment and capital-producer rents, in each country.
3. **Government.** Apply the fiscal rule to the surviving stock, compute the coupon, and forward-integrate the debt stock to B' .
4. **Balance sheets.** Compute asset payoffs $\mathcal{X}_X$, gross wealth $n_X^g = \mathcal{X}_X - P_X$, end-of-period assets $\mathcal{A}_X$, net worth n_X , deposits, and the next-period obligation P'_X .
5. **Continuations.** Build the next-period state vector $\mathbf{S}'$ for every quadrature node, clip it into the box, and evaluate *all* rules in each regime (`cont.eval_all`).
6. **Branch kernels.** Form $\Lambda^{(d')}$, $\varphi^{(d')}$ and hence $\Omega^{(d')}$ on each branch; take the three-branch expectations.
7. **Banker block.** The closed-form multiplier μ_X , the franchise value φ_X , bond prices q^X , the lending rate r_X^{wc} .
8. **Household block.** Dividends, income, consumption from the budget, the deposit Euler.
9. **Trade.** Import demand, net exports.
10. **Assemble** the seven residuals and return them together with a dictionary `out` of every derived object, which becomes the next iterate's rule values.

The returned pair `(res, out)` is the reason one function serves both purposes: the solver consumes `res`, and the sweep stores `out`.

6.4 The occasionally-binding constraint inside the residual

The multiplier is not a Newton unknown. It is computed *inside* the residual evaluation from Proposition 1 of the companion document,

$$\mu_X = \min \left\{ \max \left\{ 1 - \frac{\mathbb{E}[\Omega_X] (1 + r_X) n_X}{\lambda \mathcal{A}_X}, 0 \right\}, \bar{\mu} \right\}, \quad \bar{\mu} = 0.95, \quad (6.2)$$

and the capital Euler (rows 1–2 of Table 4) is then the equation that pins K' . This is the recursive-stable form of the same KKT system that the older path-based solver imposed as a Fischer–Burmeister complementarity,

$$\Psi^{FB}(\mu, \text{slack}) = \mu + \text{slack} - \sqrt{\mu^2 + \text{slack}^2} = 0 \iff \mu \geq 0, \text{ slack} \geq 0, \mu \cdot \text{slack} = 0. \quad (6.3)$$

Both encode (1.3) exactly. The difference is numerical: the FB form is a smooth-except-at-the-origin residual added to a stacked system, whereas (6.2) is an explicit, bounded formula evaluated pointwise.


```

1  # closed-form mu in [0, mu_cap]: the lower max is Bocola's KKT switch (hard, as in
2  # his residual_model.m -- it IS the complementarity, not a numerical guard); the
3  # upper cap keeps alpha = E_0m R/(1-mu) finite when a default-regime net worth n
4  # goes negative, and is SMOOTHED because unlike the KKT switch it is a guard that
5  # would otherwise plant a plateau in a fitted object.
6  _MU_CAP = 0.95
7  mu_D = float(_smin(max(1.0 - E_0m_D * (1.0 + rdep_D) * n_D / lev_D, 0.0),
8  _MU_CAP, _GUARD_EPS))

```

Listing 11: The KKT switch (hard) inside a smoothed cap. `point_map.py`.

★ *The distinction in that comment is the single most important line of implementation philosophy in the solver. The lower `max` is economics — it is the complementarity condition itself — and must stay hard, because smoothing it would blur the regime boundary the model is about. The upper `min` is a numerical guard against a diverging recursion, and must be smoothed, because a hard cap plants a plateau inside an object that is subsequently fitted by a global polynomial. Kinks that are economics stay; kinks that are numerics get smoothed. See §14.*

7 Computing expectations: quadrature over a continuum and a fork

7.1 Gauss–Hermite quadrature

Every conditional expectation in the model is an integral against the standard normal density,

$$\mathbb{E}_t[g(\varepsilon')] = \int_{-\infty}^{\infty} g(\varepsilon) \frac{1}{\sqrt{2\pi}} e^{-\varepsilon^2/2} d\varepsilon. \quad (7.1)$$

Gaussian quadrature approximates this by a weighted sum at optimally-chosen nodes,

$$\mathbb{E}_t[g(\varepsilon')] \approx \sum_{m=1}^M w_m g(\varepsilon_m), \quad (7.2)$$

where the nodes ε_m are the roots of the degree- M orthogonal polynomial for the weight function and the weights w_m are chosen to integrate low-order polynomials exactly. The defining property is remarkable: an M -node Gaussian rule is exact for all polynomials of degree $\leq 2M - 1$. With $M = 5$ nodes the rule integrates polynomials up to degree nine exactly, which for smooth integrands is far more accurate than a 5-point Monte Carlo draw (whose error falls only as $M^{-1/2}$).

7.2 The probabilists' versus physicists' trap

There are two conventions for Hermite polynomials, and confusing them silently rescales the shock.

- **Physicists' Hermite** H_n is orthogonal with respect to e^{-x^2} , so its rule approximates

$$\int g(x) e^{-x^2} dx \approx \sum_m \tilde{w}_m g(x_m).$$

Recovering an $N(0, \sigma^2)$ expectation then requires the map $\varepsilon = \sqrt{2}\sigma x_m$ and weights $\tilde{w}_m/\sqrt{\pi}$.

- **Probabilists' Hermite** He_n is orthogonal with respect to $e^{-x^2/2}$. Its nodes are already in standard-deviation units, so $\varepsilon = \sigma x_m$ with weights normalised to sum to one.

This code uses the probabilists' rule, so the natural map $s' = \text{mean} + \sigma_s x_m$ is exact:

```

1 def gh_nodes(n=7):
2     # GAUSS-HERMITE NODES/WEIGHTS FOR ONE STANDARD-NORMAL INNOVATION.
3     # hermegauss is the PROBABILISTS' rule (weight exp(-x^2/2)), so the nodes are
4     # already in standard-deviation units and s' = mean + sigma*node is exact.
5     # (Bocola's GaussHermite.m returns the PHYSICISTS' rule and then uses the same
6     # sigma*node map, which silently rescales his innovation by 1/sqrt(2).)
7     x, w = np.polynomial.hermite_e.hermegauss(n)
8     return x, w / w.sum()

```

Listing 12: Quadrature nodes. `point_map.py`, `gh_nodes`.

★ *The parenthetical is a real, documented discrepancy with the reference implementation, and it is worth understanding as a cautionary tale. Applying the $\sigma \times$ node map to physicists' nodes implicitly divides the innovation standard deviation by $\sqrt{2}$. The reference model's reported parameter is $\sigma_s = 0.63$, but the model it actually solves contains $\sigma_s^{\text{eff}} = 0.63/\sqrt{2} = 0.4455$. To reproduce those numbers exactly one must set `cal["sigma_s"] = 0.4455` here — reproducing the solved model, not the reported parameter. Two lines of quadrature convention change the effective volatility of the driving process by 29%.*

7.3 Mixing a continuous shock with a discrete fork

The expectation is not over ε' alone. Next period the sovereign either does not default ($d' = 0$), or defaults and the central-bank backstop is honoured, or defaults and the backstop is reneged. The measure is therefore a product of the Gauss–Hermite rule with a three-point discrete distribution, giving $3M$ evaluation nodes:

$$\mathbb{E}_t[g] = \sum_{m=1}^M w_m \left\{ \underbrace{(1 - \pi^d)}_{w^{\text{nd}}} g(0, \mathbf{S}'_m) + \underbrace{\pi^d \varkappa}_{w^{\text{hon}}} g^{\text{hon}}(\mathbf{S}'_m) + \underbrace{\pi^d (1 - \varkappa)}_{w^{\text{ren}}} g(1, \mathbf{S}'_m) \right\} \quad (7.3)$$

with $\pi^d = \pi^d(s_t)$ the current logistic default probability and $\varkappa$ the priced backstop-activation probability. The implementation folds the branch probabilities directly into the quadrature weight vectors, so a branch expectation is a pair of dot products:

```

1 w_nd, w_def = wq * (1.0 - pd), wq * pd
2 a_tpi = float(cal.get("tpi_activation", 0.0))
3 w_hon, w_ren = w_def * a_tpi, w_def * (1.0 - a_tpi)
4
5 def _E(v0, v1):
6     # TWO-BRANCH EXPECTATION (no-default vs default) -- F-safe legs.
7     return float(np.dot(w_nd, v0) + np.dot(w_def, v1))
8
9 def _E3(v0, vh, vr):
10    # THREE-BRANCH EXPECTATION: no-default / default-honoured / default-reneged.
11    return float(np.dot(w_nd, v0) + np.dot(w_hon, vh) + np.dot(w_ren, vr))

```

Listing 13: Branch weights and the expectation operators. `point_map.py`.

► **Exact nesting by construction.** Setting $\varkappa = 0$ makes $w^{\text{hon}} \equiv 0$ in exact floating-point arithmetic, so (7.3) collapses to the two-branch case with no residual contamination — not approximately, but bit-for-bit. Likewise $\pi^d \equiv 0$ makes $w^{\text{def}} \equiv 0$ and recovers the risk-neutral model exactly. This is what makes the nesting regression test (`tests/test_recursive_nesting.py`) a meaningful check rather than a tolerance comparison, and it is why the branch probabilities are multiplied into the weights rather than branched on with an `if`.

7.4 Evaluating the continuation

Each of the M quadrature nodes needs a full next-period state. These are assembled as a single $(M \times 7)$ array and the rules are evaluated on all of them in one vectorised call — one basis-matrix

construction and one matrix multiply per regime, rather than M scalar interpolations:

```

1  def _cont(d_next):
2      # CONTINUATION RULE VALUES + NEXT-PERIOD CAPITAL RETURN, REGIME d_next.
3      Sn = np.empty((m, 7))
4      Sn[:, IK_D] = Kp_D; Sn[:, IK_F] = Kp_F
5      Sn[:, IP_D] = Pp_D; Sn[:, IP_F] = Pp_F
6      Sn[:, IB_D] = Bp_D
7      Sn[:, IS] = s_next
8      Sn[:, IZ] = Z_next
9      r = cont.eval_all(d_next, cont.grid.clip(Sn))
10     # guard continuation outputs before they enter fractional powers / sqrt
11     # (a deep default-regime iterate can push N, p, C negative -> NaN)
12     for k in ("N_D", "N_F"):
13         r[k] = np.maximum(r[k], 0.05)
14     for k in ("C_D", "C_F"):
15         r[k] = np.maximum(r[k], 1e-3)
16     r["p"] = np.maximum(r["p"], 1e-2)

```

Listing 14: Vectorised continuation evaluation. `point_map.py`, `_cont`.

Note the ordering: only s' varies across quadrature nodes; the endogenous states K' , P' , B' are common to all M of them, because they are chosen at t and are therefore deterministic given $\mathbf{x}$. The array is broadcast accordingly.

7.5 The honoured branch as a blend

The backstop-honoured continuation is not a separately solved regime — there is no third coefficient set. It is a convex blend of the two solved regimes' rules, governed by the “real shield” parameter ς :

$$g^{\text{hon}}(\mathbf{S}') = (1 - \varsigma) g(1, \mathbf{S}') + \varsigma g(0, \mathbf{S}'), \quad (7.4)$$

implemented as a dictionary comprehension over all rules at once:

```

1  # honoured continuation = d'=1 recession blended toward d'=0 by 'shield'
2  rh = {k: (1.0 - shield) * r1[k] + shield * r0[k] for k in r1}

```

Listing 15: The honoured branch. `point_map.py`.

This is an approximation — a genuinely separate backstop regime would require a third solve — but a disciplined one: at $\varsigma = 1$ the honoured branch is exactly the no-default economy, at $\varsigma = 0$ it is exactly the default economy, and intermediate values interpolate. The baseline sets $\varsigma = 0.5$ because at $\varsigma = 1$ full activation neutralises risk entirely and tips the barely-binding constraint into a slack-branch slip.

7.6 Quadrature order

The solve uses $M = 5$ nodes (`N_GH = 5` in `recursive_experiment.py`). The order is *stamped onto the solved rule set*:

```

1  # quadrature order the rules were SOLVED under; time_iteration stamps it and
2  # every reader (IRFs, decompositions, accuracy) uses it, so a solve at n_gh=5
3  # is never read back at n_gh=7 and charged the difference as approximation error
4  self.n_gh = None

```

Listing 16: Recording the quadrature order. `decision_rules.py`, `RuleSet`.

► This is a small piece of bookkeeping that prevents a subtle and embarrassing class of bug. If rules are solved with $M = 5$ and then read back with $M = 7$, the residuals evaluated at the solution will not be zero — and the difference is pure quadrature mismatch, not approximation error. Every downstream reader (impulse responses, decompositions, the accuracy report) takes

its order from `rules.n_g`, so the reported accuracy is the accuracy of the object that was actually solved.

Part 3. Code architecture and implementation

8 Architecture

8.1 Module map

The solver is deliberately layered, with the economics kept solver-agnostic. The call structure of a production run is:

Layer	Module	Responsibility
Driver	<code>main.py</code>	Steady state $\rightarrow$ TFP $\rightarrow$ risk $\rightarrow$ TPI experiments
	<code>recursive_experiment.py</code>	Grid choice, warm-start chain, homotopy, IRF readers
	<code>tpi_recursive_experiment.py</code>	Backstop-activation comparison
Solver	<code>recursive_main.py</code>	<code>time_iteration</code> , <code>_sweep</code> , <code>solve_point</code>
	<code>point_map.py</code>	<code>point_residuals</code> : the residual $R(\mathbf{x}; d, \mathbf{S}, \Gamma)$
Approximation	<code>state_grid.py</code>	SmolyakGrid, box construction, the s process
	<code>decision_rules.py</code>	RuleSet: coefficients, transforms, evaluation
	<code>eds.py</code>	Ergodic-set grid (drop-in alternative)
Economics	<code>blocks/*.py</code>	Bank, firms, capital, trade, government, household
	<code>config/*.py</code>	Calibration and the two-stage steady-state solve
Diagnostics	<code>accuracy.py</code>	Euler errors on the ergodic set, box escapes
	<code>recursive_residual.py</code>	Per-point μ /slack/residual maps
	<code>output_decomposition.py</code>	Factor decomposition of $d \log Y$

The dependency direction is strict: `blocks/` knows nothing about grids, quadrature or solvers; `point_map.py` calls `blocks/` but does not know about time iteration; `recursive_main.py` knows about grids and residuals but contains no economics. This is what allowed the entire perfect-foresight solver stack to be deleted without touching a single economic block, and what allows EDSGrid to substitute for SmolyakGrid unchanged.

8.2 The RuleSet abstraction

A RuleSet holds, for each of the 17 rules and each of the two regimes, both the point values on the grid and the fitted coefficients. Its interface is small:

```
1 class RuleSet:
2     # COEFFICIENTS AND POINT VALUES FOR EVERY RULE IN BOTH REGIMES.
3     def __init__(self, grid):
4         self.grid = grid
5         self.vals = {k: [np.empty(grid.n), np.empty(grid.n)] for k in ALL_RULES}
6         self.coef = {k: [None, None] for k in ALL_RULES}
7         self.n_gh = None
8
9     def set_values(self, name, d, values, weights=None, ridge=0.0):
10        # SET POINT VALUES FOR ONE RULE IN ONE REGIME AND REFIT ITS COEFFICIENTS.
11        self.vals[name][d] = np.asarray(values, dtype=float).copy()
12        y = to_fit(name, self.vals[name][d])
13        if weights is None and ridge == 0.0:
14            self.coef[name][d] = self.grid.fit(y)
15        else:
16            self.coef[name][d] = self.grid.fit_weighted(y, weights, ridge)
17
18    def eval_all(self, d, x):
19        # EVALUATE EVERY RULE IN REGIME d AT POINTS x -> DICT OF ARRAYS.
20        B = self.grid.basis(x)
21        return {k: from_fit(k, B @ self.coef[k][d]) for k in ALL_RULES}
```

Listing 17: The rule container. `decision_rules.py`.

► *eval_all* builds the basis matrix once and reuses it across all seventeen rules. Since the basis evaluation is $O(n \cdot d \cdot k_{\max})$ and the subsequent projection is a single matrix-vector product per rule, this makes evaluating all rules barely more expensive than evaluating one. Given that *_cont* is called two or three times per residual evaluation, and the residual is called dozens of times per Newton solve at each of 518 node-regime pairs per sweep, this single design choice is worth an order of magnitude.

8.3 Log-space collocation

Rules are *stored* in levels but *fitted* in transformed space:

```

1 LOG_RULES = frozenset({"N_D", "N_F", "Kp_D", "Kp_F", "p", "alpha_D", "alpha_F",
2                       "Q_bD", "Q_bF", "C_D", "C_F", "A_D", "A_F"})
3 GROSS_RULES = frozenset({"rdep_D", "rdep_F", "r_wc_D", "r_wc_F"})
4
5 def to_fit(name, v):
6     # LEVELS -> THE QUANTITY ACTUALLY FITTED BY THE CHEBYSHEV COLLOCATION.
7     if name in GROSS_RULES:
8         return np.log(np.maximum(1.0 + np.asarray(v, dtype=float), _FIT_FLOOR))
9     if name in LOG_RULES:
10        return np.log(np.maximum(np.asarray(v, dtype=float), _FIT_FLOOR))
11    return np.asarray(v, dtype=float)
12
13 def from_fit(name, y):
14     # FITTED QUANTITY -> LEVELS (the inverse of to_fit).
15     if name in GROSS_RULES:
16         return np.exp(np.clip(y, -50.0, 50.0)) - 1.0
17     if name in LOG_RULES:
18         return np.exp(np.clip(y, -50.0, 50.0))
19    return y

```

Listing 18: The fit transform. `decision_rules.py`.

Three things are going on here, and each is a distinct trick.

1. **Positivity by construction.** Hours, capital, consumption, prices and valuations must be positive. A polynomial fitted to levels can go negative between nodes — especially near a kink, where Gibbs oscillation overshoots — and a negative N entering $N^{1/\nu}$ produces a NaN that propagates through every subsequent expectation. Fitting $\log x$ and exponentiating makes negativity *impossible*, which removed a whole family of hard clips that previously existed only to protect positivity.
2. **No plateaus to represent.** Those hard clips were themselves harmful: a clip is a plateau, a plateau is a kink, and the interpolant then has to represent it. Removing them improves the fit for a second, independent reason.
3. **Rates handled separately.** A net interest rate may legitimately be negative, so $\log r$ is inadmissible. Rates are therefore carried as the *gross* rate $1 + r$, which is positive — the same device Bocola uses when he parameterises policies as $\exp(\bar{x} + \gamma)$.

The ± 50 clamp on the exponent is a last-resort finiteness guard: $\exp$ overflows to infinity around 709, and one infinity poisons every expectation it enters. The range $e^{\pm 50}$ spans 10^{-22} to 5×10^{21} and is unreachable by any admissible economic value, so the clamp never binds on a converged solution — it only keeps a diverging transient iterate finite long enough to be rejected.

9 Mathematics-to-code map

Table 5 indexes every mathematical object in Parts 1–2 to the exact file, function and variable that implements it.

Table 5: Mathematics to code.

Mathematical object	Code location	Names
$T_k(x)$, recurrence (3.2)	state_grid.py	chebyshev_basis_1d(x, max_deg)
Chebyshev extrema (3.7)	state_grid.py	_level_points(i)
New points Δ_i	state_grid.py	_new_points(i)
New degrees $\mathcal{K}_i$	state_grid.py	_new_degrees(i)
Index set $\mathcal{I}(\ell)$ (4.1)	state_grid.py	_multi_indices(d, mu, mu_vec)
Sparse grid $\mathcal{H}(\ell)$ (4.2)	state_grid.py	SmolyakGrid.points, _points_unit
Basis set $\mathcal{B}(\ell)$ (4.3)	state_grid.py	SmolyakGrid.degrees
Collocation matrix Φ (3.8)	state_grid.py	_Phi, _basis_unit(u)
$c = \Phi^{-1}y$ (3.9)	state_grid.py	fit(values) via lu.solve
Weighted ridge fit	state_grid.py	fit_weighted(values, w, ridge)
Box map $u(\mathbf{S})$ (5.1)	state_grid.py	to_unit, from_unit
Box construction (5.2)	state_grid.py	build_state_box(...)
Rotation $z = R(\mathbf{S} - \bar{\mathbf{S}})$	state_grid.py	_fwd, _bwd, rot, centre
Clip (3.10)	state_grid.py	clip(x), outside(x)
Eigenbasis probe (5.3)	recursive_main.py	p_block_rotation(...)
State vector $\mathbf{S}$ (1.1)	state_grid.py	STATE_NAMES; indices IK.D...IZ
Unknowns $\mathbf{x}$ (1.2)	decision_rules.py	SOLVE7
Read-off rules	decision_rules.py	DERIVED
$\hat{g}(\mathbf{S}; c)$ (2.2)	decision_rules.py	RuleSet.eval, .eval_all
Log transform	decision_rules.py	to_fit, from_fit
Residual $R(\mathbf{x}; d, \mathbf{S}, \Gamma)$ (6.1)	point_map.py	point_residuals(S, d, x, cont, ...)
Closed-form μ (6.2)	point_map.py	mu_D, mu_F, lev_D, lev_F
Franchise value φ	point_map.py	alpha_D_new, alpha_F_new
Bond prices q^X	point_map.py	Q_bD_new, Q_bF_new
Lending spread $\lambda\mu/\mathbb{E}[\Omega]$	point_map.py	r_wc_D, wedge_sp_D
GH nodes (7.2)	point_map.py	gh_nodes(n)
Branch weights (7.3)	point_map.py	w_nd, w_hon, w_ren
Branch expectations	point_map.py	_E(v0,v1), _E3(v0,vh,vr)
Continuation states	point_map.py	_cont(d_next)
Honoured blend (7.4)	point_map.py	rh, shield
Smoothed guards	point_map.py	_smax, _smin, _sclip
Point solve	recursive_main.py	solve_point(S, d, cont, ..., x0)
One sweep	recursive_main.py	_sweep(rules, cont, ...)
Time iteration	recursive_main.py	time_iteration(rules, ...)
Warm-start chain, homo- topy	recursive_experiment.py	solve_recursive(...)
IRF reader	recursive_experiment.py	read_at(...)

Table 5, continued

Mathematical object	Code location	Names
Euler errors	<code>accuracy.py</code>	<code>simulate</code> , <code>euler_errors</code> , <code>accuracy_report</code>
EGM / distribution kernels	<code>fast_kernels.py</code>	<code>hh_backward</code> , <code>dist_forward</code>

Part 4. Computational tricks and optimisations

10 The outer algorithm: time iteration

10.1 Why time iteration rather than one big Newton

Given the parameterisation, the collocation conditions (2.5) are a system of

$$\underbrace{7}_{\text{equations}} \times \underbrace{259}_{\text{nodes}} \times \underbrace{2}_{\text{regimes}} = 3,626 \quad (10.1)$$

non-linear equations in as many unknowns. One could hand that to a Newton solver directly. The code does not, and the reason is the occasionally-binding constraint: a monolithic Newton must construct a 3626×3626 Jacobian by finite differences, and *any* node whose perturbed evaluation crosses the KKT switch $\{\mu = 0\}$ contaminates an entire column. With thousands of nodes, some node is almost always at the kink. The earlier `make_collocation_residual` approach did exactly this and was superseded.

Instead the solver uses **time iteration** (a Coleman operator): treat the previous iterate's rules as a *frozen continuation* Γ , and for each node independently solve the small 7×7 system $R(\mathbf{x}; d, \mathbf{S}, \Gamma) = 0$. Formally, define the operator

$$\mathcal{T}[\Gamma](d, \mathbf{S}) = \{\mathbf{x}^*, \text{derived}\} \quad \text{where} \quad R(\mathbf{x}^*; d, \mathbf{S}, \Gamma) = 0, \quad (10.2)$$

and iterate $\Gamma_{k+1} = \mathcal{T}[\Gamma_k]$ to a fixed point. Three advantages follow immediately:

1. each node's solve is 7×7 , so a kink at one node is confined to that node;
2. nodes are independent, so a failure is *localisable* and can be handled (§13) rather than poisoning the global step;
3. the fixed point of $\mathcal{T}$ is exactly the collocation solution, so nothing is given up.

10.2 The sweep

One sweep loops over regimes and nodes, solving each and collecting the results:

```
1 def _sweep(rules, cont, cal, ss, sproc, regimes, no_default, n_gh, keep_tol=1e-3):
2     # ONE TIME-ITERATION SWEEP: SOLVE EVERY POINT, RETURN NEW RULE VALUE ARRAYS.
3     # A point whose solve does not clear (fn > keep_tol) RETAINS the previous
4     # iterate's values -- a failed corner must never poison the continuation.
5     n = rules.grid.n
6     new = {k: {d: np.empty(n) for d in regimes} for k in STORE()}
7     wt = {d: np.ones(n) for d in regimes} # per-point fit weight (0 = failed corner)
8     worst, n_fail = 0.0, 0
9     for d in regimes:
10         for i in range(n):
11             S = rules.grid.points[i]
12             x0 = np.array([rules.vals[k][d][i] for k in SOLVE7])
13             x, out, fn = solve_point(S, d, cont, cal, ss, sproc, x0,
14                                     no_default=no_default, n_gh=n_gh, x_ss=x_ss)
15             worst = max(worst, fn if np.isfinite(fn) else 1e3)
16             if (not np.isfinite(fn)) or fn > keep_tol: # retain old values, mask fit
17                 n_fail += 1
18                 wt[d][i] = 0.0
19                 for k in STORE():
20                     new[k][d][i] = rules.vals[k][d][i]
21             else:
22                 for j, k in enumerate(SOLVE7):
23                     new[k][d][i] = x[j]
24                 for k in DERIVED:
25                     new[k][d][i] = out[k]
26     return new, worst, n_fail, wt
```

Listing 19: One time-iteration sweep. `recursive.main.py`, `_sweep`.

10.3 Damping and the convergence test

The updated values are not adopted outright. They are damped,

$$\Gamma_{k+1} = \varpi \mathcal{T}[\Gamma_k] + (1 - \varpi) \Gamma_k, \quad \varpi = 0.25, \quad (10.3)$$

and only then refitted. Damping is what keeps the fixed-point iteration stable when the operator is not a contraction — which is exactly the situation near the constraint boundary, where a small change in the continuation can move μ across zero and produce a large change in policies. The cost is speed: with $\varpi = 0.25$ the per-sweep contraction is at best $1 - \varpi = 0.75$, so a cold start needs 70–90 sweeps.

The exit test is deliberately a *conjunction*:

```
1     for k in STORE():
2         for d in regimes:
3             old = rules.vals[k][d]
4             upd = damp * new[k][d] + (1.0 - damp) * old
5             change = max(change, np.max(np.abs(upd - old))
6                          / (np.max(np.abs(old)) + 1e-8))
7             weights = (np.where(wt[d] > 0.5, 1.0, fit_fw) if fit_mask else None)
8             rules.set_values(k, d, upd, weights=weights, ridge=fit_ridge)
9         if change < tol and worst < 1e-6:
10             return True, it + 1, worst, n_fail
11     return False, max_it, worst, n_fail
```

Listing 20: Damping, refit and the two-part exit test. `recursive_main.py`, `time_iteration`.

★ *The condition is `change < tol` AND `worst < 1e-6`, and `n_fail` is part of the return contract. The reason is a failure mode that occurred and went unnoticed: a sweep can look perfectly converged on `change` while a quarter of the grid is frozen on its previous values and never clearing. The rules stop moving — because the failing nodes stop moving by construction — and a change-only test declares success on a solution that does not satisfy its own equations at 25% of its collocation points. The caller must be able to see this, so the failure count is returned, and a non-converged stage prints a warning even when `verbose` is off, because the TPI and decomposition experiments call the solver with `verbose=False` and silence there is exactly how a run reporting failure into _ went unnoticed.*

11 The inner solver

11.1 Powell’s hybrid method

Each node’s 7×7 system is solved by `scipy.optimize.root(method="hybr")`, a modified Powell hybrid method (MINPACK’s `hybrd`). It is a trust-region Newton variant: it takes a Gauss–Newton step when the model is trusted and a gradient-descent step otherwise, which makes it far more robust than pure Newton to the poor initial guesses that occur early in the iteration.

The Jacobian is not supplied analytically — deriving $\partial R / \partial \mathbf{x}$ through a three-branch quadrature over interpolated continuations would be an enormous and fragile undertaking — so MINPACK builds it by forward differences with step $\sqrt{\epsilon_{\text{mach}}} \approx 1.49 \times 10^{-8}$ relative to each component. **This finite-difference step size is the single number that governs every smoothing decision in §14:** a guard whose kink is narrower than 1.5×10^{-8} is invisible to the solver in a bad way — the difference quotient straddles it and returns a meaningless slope.

11.2 Dual starts and graceful failure

```
1 def solve_point(S, d, cont, cal, ss, sproc, x0, no_default=False, n_gh=7, x_ss=None):
2     # SOLVE THE 7 MARKET-CLEARING UNKNOWNNS AT ONE POINT (FROZEN CONTINUATION).
3     # hybr from the warm start (the common case: one cheap solve near the fixed
```

```

4      # point); a single fallback from the SS guess only if that misses.
5      def f(x):
6          try:
7              return point_residuals(S, d, x, cont, cal, ss, sproc,
8                                     n_gh=n_gh, no_default=no_default)[0]
9          except (ValueError, RuntimeError, FloatingPointError):
10             return np.full(7, 10.0)
11
12     sol = root(f, x0, method="hybr", tol=1e-12)
13     best = (sol.x, np.max(np.abs(sol.fun)))
14     if best[1] > 1e-9 and x_ss is not None:
15         sol2 = root(f, x_ss, method="hybr", tol=1e-12)
16         if np.max(np.abs(sol2.fun)) < best[1]:
17             best = (sol2.x, np.max(np.abs(sol2.fun)))

```

Listing 21: The point solve. `recursive_main.py`, `solve_point`.

Three techniques appear in these few lines.

Exceptions as penalties. An economically infeasible trial vector — one that makes the Jermann inversion demand a negative bracket, say — raises rather than returning a NaN. The wrapper catches it and returns a large constant residual $(10, \dots, 10)$, which steers the trust region away from that region instead of crashing the sweep. The alternative, returning NaN, would poison the Jacobian.

Warm starting. The initial guess x_0 is the *previous iterate's solved values at this same node*. Near the fixed point this is an excellent guess and the solve converges in a handful of iterations. This is the single largest speed factor in the whole algorithm.

A second start, only if needed. If the warm start misses ($\|R\| > 10^{-9}$), one retry from the steady-state guess is attempted and the better of the two is kept. Attempting it unconditionally would double the cost for no benefit; not attempting it at all leaves nodes stranded when the warm start is in the wrong basin.

Finally, if *both* evaluations raise, the function returns a sentinel $\|R\| = 10^3$ rather than propagating the exception, so `_sweep` can mask the node and continue.

12 Continuation and homotopy

12.1 Breaking the simultaneity knot

There is a genuine circularity in the model: the current bond price q_t^D enters the government's financing need, which determines B' , which is part of next period's state, which determines the continuation values that price q_t^D . The same loop exists for the franchise value φ and for the working-capital rate r^{wc} (whose loan quantity depends on the wage, which depends on r^{wc} , which depends on μ , which depends on the balance sheet).

The solver cuts all three knots the same way: **take the circular object from the frozen previous iterate**.

```

1      # current-period bank valuations = FROZEN previous-iterate rules at this state
2      # (breaks the S'<->Q_b knot; coincide at the fixed point)
3      Sm = np.atleast_2d(S)
4      Q_bD_cur = float(cont.eval("Q_bD", d, Sm)[0])
5      alpha_D_cur = float(cont.eval("alpha_D", d, Sm)[0])
6      r_wc_D_cur = float(cont.eval("r_wc_D", d, Sm)[0])

```

Listing 22: Knot-breaking with frozen valuations. `point_map.py`.

► *This is the defining idea of time iteration applied within the period, and its correctness argument is simple: at the fixed point, the frozen value and the freshly computed value coincide*

by construction, so the converged solution satisfies the true simultaneous system. Away from the fixed point they differ, but that difference is just another dimension of the iteration error, which the convergence test measures. What it buys is that the 7×7 system stays 7×7 and stays solvable.

Note the internal consistency requirement this creates: the labour residual uses the *contemporaneous* r^{wc} (computed fresh from μ), while the loan *quantity* uses the frozen one. The two coincide at the fixed point; using the frozen value in both places would leave the labour condition unenforced.

12.2 Warm-start chaining across regimes

The default regime $d = 1$ cannot be cold-started: it is a deep recession far from the steady state, and a cold Newton from steady-state values does not converge at any node. The driver therefore chains:

1. Solve $d = 0$ at $\pi^d \equiv 0$ (the risk-neutral economy) from a steady-state cold start. This is the easiest problem in the family.
2. Copy the converged $d = 0$ values into the $d = 1$ slots as an initial guess.
3. Solve $d = 1$ by homotopy in the recovery rate (below).
4. Solve both regimes *jointly* with risk priced, which is the first stage in which the two regimes see each other.

12.3 Homotopy in the recovery rate

Step 3 is a continuation method. Rather than jumping to the calibrated haircut, the solver walks the recovery rate down in four steps, re-solving at each:

```

1  # d=1 (post-default) via a HAIRCUT HOMOTOPY: warm-start from d=0, then lower
2  # the recovery from ~1 (=no default, identical to d=0) down to the calibrated
3  # value in steps, re-solving each -- so the deep recession is reached by
4  # continuation, not cold-started (which does not converge).
5  for k in STORE_RULES:
6      rules.set_values(k, 1, rules.vals[k][0].copy())
7  for rec in (0.85, 0.70, 0.55, rec_target):
8      cal["recovery_rate_D"] = rec
9      ok1, it1, w1, f1 = time_iteration(rules, cal, ss, sproc, regimes=(1,),
10                                     no_default=False, damp=0.25, tol=1e-6,
11                                     max_it=80, n_gh=N_GH)

```

Listing 23: Haircut homotopy. `recursive_experiment.py`, `solve_recursive`.

Formally, let $\varrho \in [\varrho_{\text{target}}, 1]$ index a family of problems $\mathcal{P}(\varrho)$, with $\mathcal{P}(1)$ the trivial no-haircut problem whose solution is known (it is the $d = 0$ solution) and $\mathcal{P}(0.45)$ the one wanted. The homotopy traces the solution path $\varrho \mapsto \Gamma^*(\varrho)$ by using the solution at each step as the initial guess for the next. It works whenever the path is continuous and the steps are small enough that each solution lies in the next problem's basin of attraction.

► *Homotopy is the standard remedy when a non-linear solve fails from a cold start but the problem embeds in a family with a known easy member. The recovery rate is the natural parameter here precisely because $\varrho = 1$ makes the default branch identical to the no-default branch — a perfect known solution at the end of the path, not merely an approximate one.*

13 Robustness of the fit

13.1 Failed nodes must not enter the fit

Some collocation nodes have no equilibrium at all — in the default regime, corners that combine a deep haircut with already-low net worth are genuinely infeasible. Since the fit (3.9) is a *global* least-squares/interpolation problem, a single garbage value at one node moves the polynomial *everywhere*, including at the ergodic centre where the answer is read. Measured: one saturated node moves the interpolant at the ergodic centre by about 26% and can drive it negative.

Two defences are layered. First, a failed node retains its previous values, so the value entering the fit is at least economically sensible. Second, the fit itself can be switched from exact square collocation to a masked, ridge-regularised weighted least squares:

```

1 def fit_weighted(self, values, w=None, ridge=0.0):
2     # RIDGE-REGULARISED WEIGHTED LEAST-SQUARES FIT: the fix for global-fit corner
3     # poisoning at mu=2. w in [0,1] per point (0 = point EXCLUDED from the fit, so
4     # an unsolvable/frozen corner cannot leak into the coefficients); the ridge
5     # penalises high-degree coefficients (damps the Gibbs wiggle at the kink) with
6     # a small uniform floor for invertibility when points are masked.
7     values = np.asarray(values, dtype=float)
8     if w is None and ridge == 0.0:
9         return lu_solve(self._lu, values)
10    w = np.ones(self.n) if w is None else np.asarray(w, dtype=float)
11    A = self._Phi.T * w                                # Phi^T @ diag(w)
12    lhs = A @ self._Phi                                # Phi^T W Phi
13    diag_scale = float(np.mean(np.diag(lhs))) + 1e-12
14    td = self.degrees.sum(axis=1).astype(float)        # total degree per basis fn
15    reg = diag_scale * (ridge * td / max(td.max(), 1.0) + 1e-6)
16    lhs[np.diag_indices_from(lhs)] += reg
17    return np.linalg.solve(lhs, A @ values)

```

Listing 24: Robust weighted ridge fit. `state_grid.py`, `fit_weighted`.

The estimator is

$$c = \left(\Phi^\top W \Phi + \Lambda \right)^{-1} \Phi^\top W y, \quad \Lambda_{bb} = \bar{\Phi} \left(\rho \frac{|k_b|_1}{\max_b |k_b|_1} + 10^{-6} \right), \quad (13.1)$$

with $|k_b|_1 = \sum_j k_{b,j}$ the total degree of basis function b . Two design details are worth noting.

Degree-graded ridge. The penalty is proportional to the total degree, so low-order (economically meaningful) terms are barely penalised while high-order terms — exactly those that produce Gibbs oscillation near a kink — are damped hard. A uniform ridge would shrink the level and slope of the rule, which is not the problem.

Soft rather than hard masking. The default failure weight is 0.1, not 0. Hard masking (weight zero) collapses the fit when a sweep has many failures — $\Phi^\top W \Phi$ loses rank — so failed corners are kept as weak anchors: they still constrain the fit enough to keep it full-rank and sane, but their poisoning is cut roughly tenfold. The 10^{-6} uniform floor guarantees invertibility regardless.

14 Smoothing, guards and bounds

14.1 The smoothing primitives

Wherever a bound must be imposed on a quantity that is subsequently *fitted* or that enters a finite-difference Jacobian, the hard operator is replaced by its smooth counterpart:

$$\text{smax}(x, \underline{x}; \epsilon) = \underline{x} + \frac{1}{2} \left[(x - \underline{x}) + \sqrt{(x - \underline{x})^2 + \epsilon^2} \right], \quad (14.1)$$

$$\text{smin}(x, \bar{x}; \epsilon) = \bar{x} - \frac{1}{2} \left[(\bar{x} - x) + \sqrt{(\bar{x} - x)^2 + \epsilon^2} \right]. \quad (14.2)$$

```

1 def _smax(x, floor, eps=1e-3):
2     # SMOOTH MAX(x, floor) -- differentiable everywhere so the FD-Jacobian stays valid.
3     return floor + 0.5 * ((x - floor) + np.sqrt((x - floor) ** 2 + eps ** 2))
4
5 def _sclip(x, lo, hi, eps=None):
6     # SMOOTH clip: the guards below must not be plateaus. A hard np.clip on a value
7     # that is then FITTED puts a kink inside the box, and one saturated node moves the
8     # interpolant at the ergodic centre by ~26% and can drive it negative (measured).
9     return _smin(_smax(x, lo, eps or _GUARD_EPS), hi, eps or _GUARD_EPS)

```

Listing 25: Smoothing primitives. `point_map.py`.

14.2 Calibrating ϵ : the two-sided argument

The smoothing scale is not arbitrary. It is squeezed between two requirements.

Upper bound (bias). Away from the bound, `smax` differs from `max` by approximately $\epsilon^2/(4 \text{gap})$, where “gap” is the distance to the bound. Since the steady state must remain an exact rest point of the recursive system to within the acceptance tolerance, ϵ must be small enough that this bias is negligible there. At $\epsilon = 10^{-5}$ the induced steady-state error is about 10^{-11} — three orders of magnitude below acceptance.

Lower bound (visibility). The smoothing exists to hide the kink from the finite-difference Jacobian, whose step is $\approx 1.5 \times 10^{-8}$ (§11). If ϵ were comparable to or smaller than that step, the difference quotient would straddle the smoothed region and see the kink anyway. At $\epsilon = 10^{-5}$ the smoothing is roughly three decades wider than the FD step.

$$\underbrace{1.5 \times 10^{-8}}_{\text{FD step}} \ll \underbrace{\epsilon = 10^{-5}}_{\text{_GUARD_EPS}} \ll \underbrace{\sqrt{4g\tau}}_{\text{bias limit}}. \quad (14.3)$$

The net-worth floor keeps its own larger $\epsilon = 10^{-3}$, because it operates on a quantity of order n_{ss} rather than on an $O(1)$ ratio.

14.3 Inventory of guards

★ *The cap interaction in row three is a real bug that was found and fixed, and it illustrates why guards must be designed jointly. An older hard clip of φ at 8.0 bound first — at $\mu \approx 0.76$ — and bit exactly where the net-worth floor puts the deep default corner: $n = 0.15 n_{ss}$ implies $\mu \approx 0.85$ and $\varphi \approx 13$, and the reference solution legitimately reaches $\varphi/\bar{\varphi} = 13.6$. Two guards intended to be slack were in fact contradicting each other and truncating a region the model genuinely visits.*

15 Vectorisation and low-level performance

15.1 Where the time goes

A production risk-stage solve is roughly 90 minutes: 259 nodes $\times$ 2 regimes $\times$ ~ 100 sweeps $\times$ ~ 30 residual evaluations per Newton solve ≈ 1.5 million residual evaluations, each containing two or three continuation evaluations of 17 rules at 5 quadrature nodes. The optimisations below target that innermost product.

Table 6: Every bound in the residual, and its status.

Guard	Value	Status and rationale
KKT switch $\max\{\cdot, 0\}$ on μ	—	Hard. It is the complementarity condition, i.e. economics. Smoothing it would blur the regime boundary.
μ cap	0.95	Smoothed. Keeps $\varphi = \mathbb{E}[\Omega]R/(1 - \mu)$ finite when default-regime net worth goes negative.
φ cap	40	Smoothed. The φ recursion has slope $\beta(1 - f)R/(1 - \mu)$, which exceeds one once $\mu > 1 - \beta(1 - f)R \approx 0.04$; above that the cap is what arrests a divergent fixed point, not cosmetics. Tunable via <code>cal["alpha_cap"]</code> .
Net-worth floor	$0.15 n_{ss}$	Smoothed , $\epsilon = 10^{-3}$. Bocola's own $\max\{\cdot, 0.65\}$ device. Inactive in the ergodic region (<code>nw_floor_frac</code> = 0 reproduces the baseline); catches only deep default corners.
Bond price bounds	$[0.2, 1.2]$	Smoothed. Numerical backstops, never binding in the ergodic region (the reference solution bottoms at 0.639).
Consumption bounds	$[0.3, 2.0] \times C_{ss}$	Smoothed , with ϵ scaled by C_{ss} so the relative tightness matches the $O(1)$ guards.
Continuation floors	$N \geq 0.05, C \geq 10^{-3}, p \geq 10^{-2}$	Hard — but applied to continuation <i>inputs</i> before fractional powers, not to fitted outputs, so no plateau is collocated.
Jermann bracket	> 0	Raises. An infeasible investment rate throws <code>ValueError</code> , which the solver wrapper converts into a penalty residual.
Exponent clamp	± 50	Hard. Finiteness only; unreachable by admissible values.

15.2 Vectorisation

- **Basis matrices, not scalar interpolations.** `eval_all` builds one $(M \times n)$ basis matrix and applies it to all 17 coefficient vectors (§8).
- **One LU factorisation for all fits.** Φ is factorised at grid construction; every one of the 34 fits per sweep is a back-substitution.
- **Quadrature as dot products.** Branch probabilities are folded into the weight vectors so an expectation is `np.dot`, with no Python-level loop over nodes or branches.
- **Broadcast continuation states.** Only s' varies across quadrature nodes; the endogenous states are assigned by broadcast into the $(M \times 7)$ array.
- **Single bincount in the distribution step.** The Young lottery scatter is done with one `np.bincount` over flattened (a, e) indices rather than a per-state `np.add.at` — roughly $4\times$ faster.

15.3 JIT compilation with an exact fallback

The two hot loops of the heterogeneous-agent block — EGM backward induction and the distribution forward iteration — are inherently sequential and cannot be vectorised away. They are compiled with numba:

```
1  try:
2      from numba import njit
3      HAVE_NUMBA = True
4  except ImportError:
5      HAVE_NUMBA = False
6      def njit(*args, **kwargs):
7          # NO-OP STAND-IN FOR numba.njit WHEN NUMBA IS UNAVAILABLE.
8          def wrap(f):
9              return f
10         return wrap
11
12  @njit(cache=True)
13  def hh_backward(a_grid, Pi_T, r_path, y_path, c_terminal, beta, sigma, a_min, vN_path):
14      # EGM BACKWARD INDUCTION OVER THE PATH (MIRRORS household.egm_step EXACTLY).
```

Listing 26: JIT with a transparent fallback. `fast_kernels.py`.

► Three details make this safe. (i) The decorator is stubbed when `numba` is absent, so the module imports and runs identically without the dependency — slowly, but correctly. (ii) The kernels replicate the `numpy` reference operation for operation, and the equivalence is regression-tested (`tests/test_fast_kernels.py`), so the fast path can never silently diverge from the reference semantics. (iii) `cache=True` persists the compiled artefacts, which matters because spawned worker processes would otherwise each pay the JIT compilation cost.

Note in particular that `np.searchsorted` is hand-written as an explicit binary search inside the JIT kernel, because `numba`’s coverage of `numpy`’s search routines does not extend to the exact call signature used; the hand-rolled version is what makes the kernel compile in `nopython` mode.

15.4 The spawn guard

On macOS, Python’s `multiprocessing` uses the *spawn* start method, which re-imports the parent module in every worker. Any unguarded top-level code therefore runs once per worker. This produced a real and initially baffling symptom — “SS solved” printed nine times — and the rule is now absolute: every standalone script that may fan out is wrapped in


```

1 if __name__ == "__main__":
2     main()

```

The recursive solver's pointwise solves are serial, so the hazard is currently latent; the guard is kept regardless, since the modules are importable and the cost of the guard is zero.

16 Diagnosing the solution

16.1 Why residuals at the nodes are not enough

By construction, collocation drives R to zero *at the nodes*. Reporting that $\max_b \|R(\mathbf{S}_b)\| = 10^{-14}$ therefore says almost nothing about solution quality — it is close to a tautology. The meaningful question is how large the residual is where the model actually spends its time, which is generally *not* at the collocation nodes.

16.2 Euler errors on the ergodic set

The diagnostic simulates the solved rules forward under drawn innovations and, at every visited state, evaluates the seven equilibrium conditions *at the rule-implied policy* — with no re-solving. The distribution of $\log_{10} |R|$ per equation is the accuracy report.

```

1 for t in range(T + burn):
2     Sm = np.atleast_2d(S)
3     x = np.array([float(rules.eval(k, 0, Sm)[0]) for k in SOLVE7])
4     _, o = point_residuals(S, 0, x, rules, cal, ss, sproc, n_gh=ngh, ...)
5     ...
6     Sn = S.copy()
7     Sn[0], Sn[1] = x[2], x[3] # K' = Kp
8     Sn[2], Sn[3] = o["Pp_D"], o["Pp_F"]
9     Sn[4] = o["Bp_D"]
10    Sn[5] = ((1.0 - sproc["rho_s"]) * sproc["s_star"] + sproc["rho_s"] * S[5]
11            + sproc["sigma_s"] * rng.standard_normal())
12    S = rules.grid.clip(Sn)[0]

```

Listing 27: Simulating the solved rules. `accuracy.py`, `simulate`.

Note that the endogenous states are advanced by the *period map's own* end-of-period values, so the simulated path is the model's, not the grid's.

16.3 Box escapes

The same routine reports how often, and by how much, the simulated path leaves the collocation box — per state dimension, as a fraction of box width:

```

1 def outside(self, x):
2     # PER-DIMENSION BOX VIOLATION IN ROTATED COORDS, AS A FRACTION OF BOX WIDTH.
3     z = self._fwd(x)
4     return np.maximum(np.maximum(self.lo - z, z - self.hi), 0.0) / (self.hi - self.lo)

```

Listing 28: Reporting box violations. `state_grid.py` and `accuracy.py`.

► This turns the box-invariance discussion of §5 from theory into a measurement. States are deliberately not clipped before being recorded — leaving the box is a result to be reported, not a condition to be silently enforced. If the ergodic path spends a material fraction of periods outside the box, the reported impulse responses are partly extrapolation, and the bands must widen.

16.4 Reading impulse responses off the rules

There is one subtlety in how results are read, and it is worth stating because the obvious alternative is wrong.

```

1 def read_at(rules, cal, ss, sproc, S):
2     # READ THE CONVERGED RULES AT STATE S (binding branch), returning the implied
3     # allocation + the point residual there (accuracy). Evaluating the period map
4     # at the rules' OWN policy values stays on the binding branch -- re-solving
5     # with a root finder can slip onto the nearby slack equilibrium at the barely-
6     # binding SS. This is the standard way to read a global solution's IRF.

```

Listing 29: Reading the solution, not re-solving it. `recursive_experiment.py`, `read_at`.

★ *Re-solving at each impulse-response date is tempting — it would drive the labour-FOC residual to zero — but it is wrong at a barely-binding steady state. Measured at $\ell = 1$, the Newton step slips onto the slack equilibrium in 49 of 60 periods: μ_D walks from 0.103 to 0 and the entire path turns expansionary ($Y_D + 2.5\%$ on impact against -0.004% for the rule read). The residual seen when reading the rules is genuine approximation error, and the cure is a finer grid — not re-solving the current period, which changes which equilibrium branch is selected.*

17 The known failure: why $\ell = 2$ does not converge

Documenting what does *not* work is part of the specification. The production solve runs at $\ell = 1$ with anisotropic refinement in s ; the isotropic $\ell = 2$ grid does not converge, and the reasons are understood.

Reason 1: the multiplier loses its bite. At $\ell = 2$ the sign of the output response can flip. The mechanism is that a coarser or differently-weighted representation lets $\mathbb{E}[\Omega]$ rise where it should fall, which by (6.2) drives $\mu \rightarrow 0$ — the constraint goes slack precisely when it should bind. A separate study established that binding harder at the steady state (raising f and the target spread so that $\bar{\mu} = 0.057$) fixes the sign, but that this is a calibration change, not a numerical fix, and it was not adopted.

Reason 2: corner poisoning. Even with the sign repaired, the joint residual remains of order one. At $\ell = 2$ some 40–50% of collocated corners are infeasible: a wide box is forced by the near-unit-root capital states, and the post-default $d = 1$ regime simply has no equilibrium at its cross-corners. Those points cannot be fitted, tightened, or masked away in sufficient number — with that many failures, hard masking collapses the fit and soft masking still leaks.

The structural fix. The corners must not be *collocated* at all, which is what the ergodic-set grid of `eds.py` implements: fit on the simulated cloud, where every point is feasible by construction, using a low complete-degree basis that cannot produce Gibbs oscillation. This is the documented next step rather than a completed result.

► *The general lesson transfers to any projection solve of a model with an occasionally-binding constraint and a discrete regime: **a global polynomial basis is only as good as the feasibility of its collocation set**. Refining the grid increases accuracy only if the added points are points at which the model has a solution. Where they are not, refinement makes things strictly worse — which is the opposite of the intuition carried over from one-dimensional numerical analysis.*

A Reproducing the results

All commands are run from `code/global/` with plain `python3` (numpy, scipy, matplotlib; numba optional).

Command	What it does
<code>python3 main.py</code>	Full pipeline: steady state, TFP experiment, sovereign-risk pass-through, accuracy report, TPI activation overlay. ~ 2 h.
<code>python3 -m solver_recursive.recursive_experiment</code>	Sovereign-risk stage only.
<code>python3 -m solver_recursive.tpi_recursive_experiment</code>	OMT/TPI activation comparison at 0/50/100%.
<code>python3 tests/test_state_grid.py</code>	Smolyak counts, collocation exactness, quadratic exactness, box coverage of the s process, rotated-box round trip.
<code>python3 tests/test_recursive_nesting.py</code>	Steady state is a rest point; $\pi^d = 0$ nests the risk-neutral model.
<code>python3 tests/test_fast_kernels.py</code>	numba/numpy kernel equivalence.

The principal solver switches live at the top of `main.py`: `MU` (TFP grid level), `RISK_MU_VEC` (the anisotropic risk grid), `NW_FLOOR`, `ROTATE_P`, `RUN_TPI` and `TPI_ACTIVATIONS`.

B Glossary of numerical terms

Term	Meaning in this document
Projection method	Replace an unknown function by a finite basis expansion and impose the functional equation in a weighted-residual sense.
Collocation	The projection variant that sets the residual to zero exactly at n chosen nodes.
Chebyshev extrema	The Gauss–Lobatto nodes (3.7); nested, endpoint-inclusive.
Smolyak grid	Sparse union of small tensor products under a total-level budget (4.1).
Approximation level ℓ	The Smolyak budget (<code>mu</code> in code); higher means more points and higher degree.
Anisotropic level ℓ_j	Per-dimension budget (<code>mu_vec</code>); concentrates resolution on chosen states.
Time iteration	Fixed-point iteration on decision rules using the previous iterate as a frozen continuation (a Coleman operator).
Frozen continuation	The previous iterate’s rules, held fixed while the current period is solved; coincides with the solution at the fixed point.
Homotopy / continuation	Solving a hard problem by tracing a path from an easy member of a parameterised family.
Damping	Convex combination of the new and old iterates, $\varpi\mathcal{T}[\Gamma] + (1 - \varpi)\Gamma$.
Gauss–Hermite quadrature	Optimal node/weight rule for Gaussian integrals; exact for polynomials of degree $\leq 2M - 1$.
Ridge / Tikhonov	Penalised least squares; here graded by basis total degree to damp high-order oscillation.
Gibbs oscillation	Spurious ringing of a global polynomial near a kink or discontinuity.
Euler error	Residual of an equilibrium condition evaluated at the approximate policy; the standard accuracy metric.
EDS grid	ε -distinguishable set: a well-spaced subsample of the simulated ergodic cloud, used as collocation points.